

Module 1 – Assignment 1.3: Basic Comparison of Relational vs. NoSQL Databases

In relational database management systems, relationships represent the logical associations created between tables to connect related data entities (Winks, 2023). Relational architecture relies on primary keys and foreign keys to establish these structural connections (Winks, 2023). A primary key uniquely identifies a single record within a table, while a foreign key in a secondary table references that primary key (Winks, 2023). This design enforces referential integrity across the database, ensuring that child entries remain tied to valid parent records (Rowy, 2023). Relationships prevent orphan records and eliminate redundant data storage across the system (Rowy, 2023).

One primary type of connection is the one-to-many relationship, where a single record in one table links to multiple entries in another table (Amazon Web Services, 2024). A common operational example occurs in e-commerce platforms, where one customer account connects to multiple order records (Amazon Web Services, 2024). Every purchase order points back to a single customer ID, but that customer can place numerous distinct orders over time (Amazon Web Services, 2024). Another essential connection is the many-to-many relationship, in which multiple entries in a table associate with multiple entries in a second table (Winks, 2023). Because relational engines cannot map this association directly between two tables, database architects use a junction table containing foreign keys from both primary tables (Winks, 2023). University course registration serves as a clear illustration of this model, where a student registers for several individual courses and each course contains a class roster populated by many enrolled students (Winks, 2023).

Evaluating database paradigms highlights distinct structural advantages depending on application goals. Relational databases excel at ensuring strong data consistency through

complete adherence to transactional ACID properties, which guarantee that operations complete successfully or roll back entirely (Microsoft, 2022). They also utilize Structured Query Language, enabling engineers to write queries that aggregate data across disparate tables without duplicating records (Rowy, 2023). On the other hand, NoSQL databases offer exceptional flexibility by supporting schema-less data storage (Adela, 2025). Developers can insert complex semi-structured documents without declaring a rigid table layout beforehand, allowing applications to adapt smoothly as data models evolve (Adela, 2025). NoSQL architectures also deliver superior horizontal scalability, distributing incoming data workloads across multi-server clusters to sustain high operational velocity (Microsoft, 2022).

Despite these benefits, both database categories carry notable drawbacks. Relational systems present structural rigidity, requiring complex schema migrations whenever application requirements shift (Winks, 2023). They also face significant scaling impediments, because expanding capacity usually requires vertical hardware upgrades that become increasingly expensive and bounded by physical hardware caps (Amazon Web Services, 2024). Conversely, NoSQL databases often compromise on immediate consistency in favor of availability, adopting eventual consistency models that can temporarily display outdated information across distributed partitions (Microsoft, 2022). NoSQL systems also lack a universal querying standard, which forces developers to learn vendor-specific APIs and introduces software lock-in risks when migrating between systems (Adela, 2025).

MySQL includes specific features that reinforce its status as a leading relational database management system. One foundational feature is the InnoDB storage engine, which provides full ACID-compliant transactional capabilities and declarative foreign key enforcement to safeguard relational structure (Adela, 2025). Developers rely on InnoDB to process complex multi-step

transactions in environments demanding immediate consistency, such as financial record-keeping (Adela, 2025). A second notable feature of MySQL is its robust secondary indexing system using B-trees (Amazon Web Services, 2024). This indexing engine allows database administrators to accelerate lookup speed on frequently queried table columns, enabling rapid execution of complex joins across extensive tabular datasets (Amazon Web Services, 2024).

MongoDB offers distinct features designed specifically for dynamic, document-centric application demands. Its core storage mechanism utilizes Binary JSON, known as BSON, to store semi-structured documents within flexible collections (Amazon Web Services, 2024). This architecture allows applications to nest arrays and sub-documents directly inside a single record, removing the requirement for multi-table joins when managing hierarchical data structures (Amazon Web Services, 2024). Another crucial feature of MongoDB is native horizontal sharding (Adela, 2025). Sharding automatically splits large collection datasets across separate nodes in a distributed cluster based on a designated shard key (Adela, 2025). Organizations deploy sharding to maintain high write throughput and low query latency across massive datasets (Adela, 2025).

References

Adela. (2025, April 24). *MongoDB vs MySQL: A complete comparison in 2025*. Bytebase.

<https://www.bytebase.com/blog/mongodb-vs-mysql/>

Amazon Web Services. (2024). *MongoDB vs. MySQL: Difference between database management systems*. AWS.

<https://aws.amazon.com/compare/the-difference-between-mongodb-vs-mysql/>

Microsoft. (2022, April 7). *Relational vs. NoSQL data*. Microsoft Learn.

<https://learn.microsoft.com/en-us/dotnet/architecture/cloud-native/relational-vs-nosql-data>

Rowy. (2023). *RDBMS vs NoSQL: How to choose in 2023*. Rowy.

<https://www.rowy.io/blog/rdbms-vs-nosql>

Winks, E. (2023, November 10). *Relational database vs NoSQL: 15 key differences*. Atlan.

<https://atlan.com/relational-database-vs-nosql/>